

Table of Contents

1. ThreadLocal是什么？	
2. ThreadLocal的具体实现	
2.1 ThreadLocal结构	
2.2 具体实现	
2.3 引用关系	
2.3.1 threadLocal引用关系	
2.3.2 扩展	
3. 源码解读	
3.1 从set方法开始	
3.2 ThreadLocal中的get()	
3.3 ThreadLocalMap中的remove()	
4. 内存泄露	
4.1 ThreadLocal 内存泄漏的原因	
4.2 ThreadLocal正确的使用方法	
4.3 总结	
5. 使用场景	
5.1 场景一：代替参数的显式传递	
5.2 场景二：解决线程安全问题	
5.3 场景三：全局存储用户信息	
6. 实战	
6.1 代码	
6.2 异常处理	
6.2.1 feign调用丢失请求头	
6.2.2 多线程中丢失请求头	
7. 扩展	
7.1 InheritableThreadLocal	
7.2 TransmittableThreadLocal(TTL)	

1. ThreadLocal是什么？

从名字我们就可以看到 `ThreadLocal` 叫做本地线程变量，意思是说，`ThreadLocal` 中填充的的是当前线程的变量，该变量对其他线程而言是封闭且隔离的，`ThreadLocal` 为变量在每个线程中创建了一个副本，这样每个线程都可以访问自己内部的副本变量。

简单实用

```
@Test
public void testThreadLocal() {
    ThreadLocal<String> local = new ThreadLocal<>();

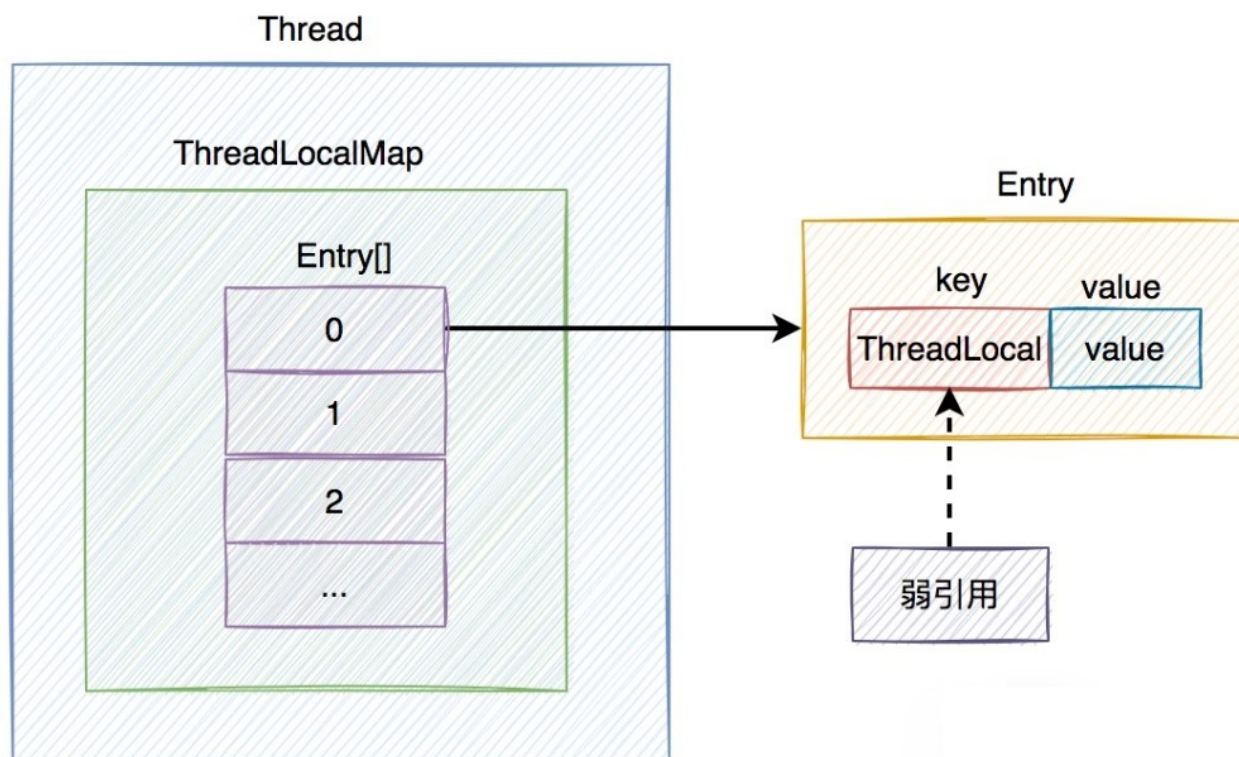
    IntStream.range(0, 10)
        .forEach(i -> new Thread(() -> {
            local.set(Thread.currentThread().getName() + ":" + i);
            System.out.println("线程: " +
Thread.currentThread().getName() + ",local:" + local.get());
        }).start()
    );
}
```

输出结果:

```
线程: Thread-0,local:Thread-0:0
线程: Thread-1,local:Thread-1:1
线程: Thread-2,local:Thread-2:2
线程: Thread-3,local:Thread-3:3
线程: Thread-4,local:Thread-4:4
线程: Thread-5,local:Thread-5:5
线程: Thread-6,local:Thread-6:6
线程: Thread-7,local:Thread-7:7
线程: Thread-8,local:Thread-8:8
线程: Thread-9,local:Thread-9:9
```

2. ThreadLocal的具体实现

2.1 ThreadLocal结构



1. 每一个线程都有一个 ThreadLocalMap ；
2. 该 Map 底层由 Entry 数组构成，含有多个 Entry ；
3. Entry 中 key 为 ThreadLocal 的弱引用， value 为我们保存的值

线程下的threadLocalMap

```
public class Thread implements Runnable {  
  
    /* ThreadLocal values pertaining to this thread. This map is maintained  
     * by the ThreadLocal class. */  
    ThreadLocal.ThreadLocalMap threadLocals = null;  
  
    /*  
     * InheritableThreadLocal values pertaining to this thread. This map is  
     * maintained by the InheritableThreadLocal class.  
     */  
    ThreadLocal.ThreadLocalMap inheritableThreadLocals = null;  
  
}
```

2.2 具体实现

从结构可以看出, 每个线程在向ThreadLocal里塞值的时候, 其实都是向自己所持有的ThreadLocalMap里塞入数据; 读的时候同理, 首先从自己线程中取出自己持有的ThreadLocalMap, 然后再根据ThreadLocal引用作为key取出value, 基于以上描述, ThreadLocal实现了变量的线程隔离 (当然, 毕竟变量其实都是从自己当前线程实例中取出来的)。

所以在上面的案例中, 会生成10个map, 每个map中就只有一个元素, key都是 `local` 变量, value是线程名称

由此看出, threadLocal底层使用map结构存储信息, key为当前的线程下的threadLocal对象, value是我们的业务值

它与hashMap有很多相似之处, 比如 扩展因子, 初始化大小等等, 但是threadLocal解决hash冲突使用的**线性探测**

解决哈希冲突的四种方法

1.开放地址方法(再散列法)

可以通俗理解为所有的地址都对所有的数值开放，而不是链式地址法的封闭方式，一个数值固定在一个索引地址位置。 $p1 = \text{hash}(\text{key})$ 如果冲突就在 $p1$ 地址的基础上+1或者散列处理， $p2 = \text{hash}(p1) \dots$

(1) 线性探测

按顺序决定值时，如果某数据的值已经存在，则在原来值的基础上往后加一个单位，直至不发生哈希冲突。

(2) 再平方探测

按顺序决定值时，如果某数据的值已经存在，则在原来值的基础上先加1的平方个单位，若仍然存在则加1的平方个单位。随之是2的平方，3的平方等等。直至不发生哈希冲突。

和线性探测相比就是改变探测了步长。因为如果都是+1来探测在数据量比较大的情况下，效率会很差。

2.链式地址法

对于**相同的值，使用链表进行连接**。使用数组存储每一个链表。（HashMap的哈希冲突解决方法）

3.建立公共溢出区

建立公共溢出区存储所有哈希冲突的数据。

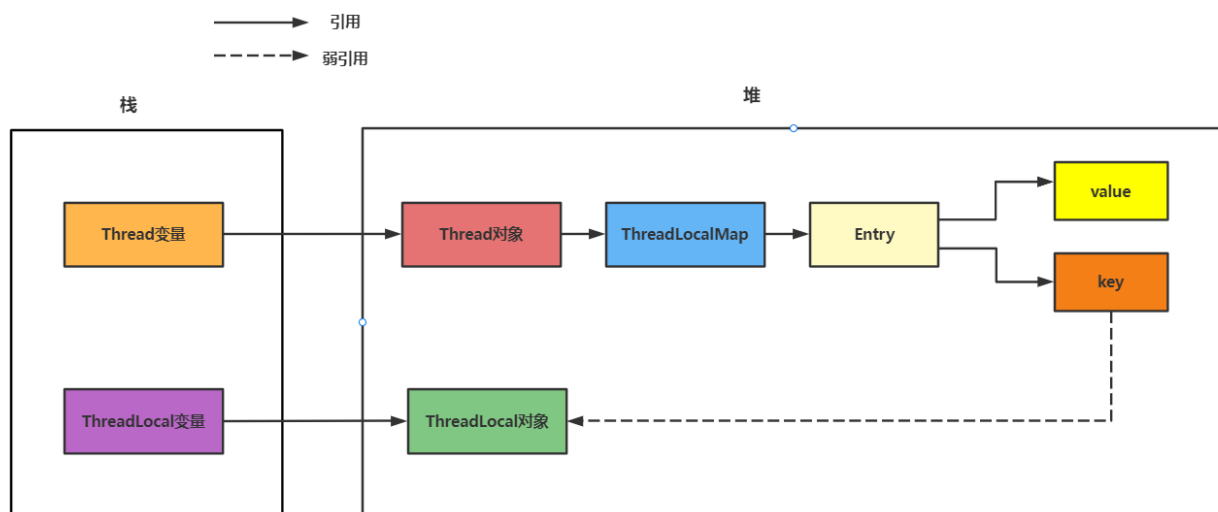
4.再哈希法

对于冲突的哈希值再次进行哈希处理，直至没有哈希冲突。

[一文理解哈希冲突四种解决方法 - 简书 \(jianshu.com\)](http://jianshu.com)

2.3 引用关系

2.3.1 threadLocal引用关系



引用关系如上图所示：

在整个引用链路中，只有 ThreadLocal 是采用了弱引用的方式进行声明的。

2.3.2 扩展

Java 中引用有四种方式。

强引用：通过 `new` 关键字产生的引用关系，无论任何情况下，只要强引用关系还存在，垃圾收集器就永远不会回收被引用的对象；

软引用：`SoftReference` 内存空间不足时，发生GC, 垃圾回收器就会回收它, 可用来实现内存敏感的高速缓存。

弱引用：声明时，通过 `WeakReference` 包裹，强度比软引用更弱一些，被弱引用关联的对象只能生存到下一次垃圾收集发生为止。当垃圾收集器开始工作，无论当前内存是否足够，都会回收掉只被弱引用关联的对象。

虚引用：虚引用也称为“幽灵引用”或者“幻影引用”，它是最弱的一种引用关系。一个对象是否有虚引用的存在，完全不会对其生存时间构成影响，也无法通过虚引用来取得一个对象实例。为一个对象设置虚引用关联的唯一目的只是为了能在这个对象被收集器回收时收到一个系统通知。

Java中提供这四种引用类型主要有两个目的：

- 可以让程序员通过代码的方式决定某些对象的生命周期。
- 有利于JVM进行垃圾回收。

[jvm内存回收前提](#)

3. 源码解读

3.1 从set方法开始

因为set是线程级别的, 所有的操作都在当前线程下, 所以set操作本身就是线程安全的

`ThreadLocal.set(T value)`

```
public void set(T value) {
    Thread t = Thread.currentThread(); // 获取当前线程
    ThreadLocalMap map = getMap(t); // 获取当前线程的ThreadLocalMap
    if (map != null)
        map.set(this, value); // map不为空则调用map的set方法
    else
        createMap(t, value); // map为空则调用createMap方法
}
```

先看看 map 为空时, createMap 方法是怎么创建 map 的。

`ThreadLocal.createMap(Thread t, T firstValue)`

```
void createMap(Thread t, T firstValue) {
    // 为传入的线程实例化一个map, 传入了自身的引用
    t.threadLocals = new ThreadLocalMap(this, firstValue);
}

/**
 * 构造一个最初包含 (firstKey, firstValue) 的新映射。
 * ThreadLocalMaps 是惰性构建的, 所以我们只有在至少有一个条目可以放入时才创建一个。
 */
ThreadLocalMap(ThreadLocal<?> firstKey, Object firstValue) {
    table = new Entry[INITIAL_CAPACITY];
    int i = firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1);
    table[i] = new Entry(firstKey, firstValue);
    size = 1;
    setThreshold(INITIAL_CAPACITY);
}
```

该构造方法主要做了以下操作:

1. 创建一个默认长度的 Entry 数组

2. 计算出传入的 ThreadLocal 应在数组中的位置
3. 实例化 Entry 放到对应位置上
4. ThreadLocalMap 元素数置为1
5. 设置要调整大小的下一个值

我们看看 ThreadLocalMap 的基础信息；

```
/**
 * 初始容量 - 必须是 2 的幂次方
 */
private static final int INITIAL_CAPACITY = 16;
/**
 * table,长度必须为 2 的幂次方
 */
private Entry[] table;
/**
 * table中元素的个数
 */
private int size = 0;
/**
 * 要调整大小的下一个大小值。
 */
private int threshold; // 默认值为 0

/**
 * Entry对象继承了弱引用(当发生垃圾回收时就会回收)
 */
static class Entry extends WeakReference<ThreadLocal<?>> {
    /** The value associated with this ThreadLocal. */
    Object value;
    Entry(ThreadLocal<?> k, Object v) {
        super(k);
        value = v;
    }
}
```

主要说一下计算索引， `firstKey.threadLocalHashCode & (INITIAL_CAPACITY - 1)`。

- 关于 `& (INITIAL_CAPACITY - 1)` ,这是取模的一种方式，对于2的幂作为模数取模，用此代替 `% (2^n)`，这也就是为啥容量必须为2的冥，在这个地方也得到了解答，至于为什么可以这样这里不过多解释，原理很简单。
- 关于 `firstKey.threadLocalHashCode`：

```
private final int threadLocalHashCode = nextHashCode();

private static int nextHashCode() {
    return nextHashCode.getAndAdd(HASH_INCREMENT);
}

private static AtomicInteger nextHashCode =
    new AtomicInteger();

private static final int HASH_INCREMENT = 0x61c88647;
```

定义了一个AtomicInteger类型, 每次获取当前值并加上HASH_INCREMENT, HASH_INCREMENT = 0x61c88647, 关于这个值和 斐波那契散列 有关, 其主要目的就是为了让哈希码能均匀的分布在2的n次方的数组里, 也就是 Entry[] table 中。

常见的散列方法是取模散列, 而斐波那契散列是另一种散列方法

[从 ThreadLocal 的实现看散列算法 - 知乎 \(zhihu.com\)](#)

再看回set方法

ThreadLocalMap.set(ThreadLocal<?> key, Object value)

ThreadLocalMap使用 线性探测法 来解决哈希冲突, 线性探测法的地址增量 $di = 1, 2, \dots, m-1$, 其中, i 为探测次数。该方法一次探测下一个地址, 直到有空的地址后插入, 若整个空间都找不到空余的地址, 则产生溢出。假设当前table长度为16, 也就是说如果计算出来key的hash值为14, 如果table[14]上已经有值, 并且其key与当前key不一致, 那么就发生了hash冲突, 这个时候将14加1得到15, 取table[15]进行判断, 这个时候如果还是冲突会回到0, 取table[0], 以此类推, 直到可以插入。所以 可以把table看成一个环形数组。

在set的时候, 会去判断 entry==null 和 key==null 的哈希槽, 并删除这些槽位, 由于使用线性探测方式, 还会挪动冲突的key的位置, 使得相同key紧凑一起

```

private void set(ThreadLocal<?> key, Object value) {

    // We don't use a fast path as with get() because it is at
    // least as common to use set() to create new entries as
    // it is to replace existing ones, in which case, a fast
    // path would fail more often than not.

    Entry[] tab = table;
    int len = tab.length;
    // 获取ThreadLocal的hashCode, 计算索引位置
    int i = key.threadLocalHashCode & (len-1);
    // 该索引位置上是否有元素, 如果有元素的话就进行线性探测
    for (Entry e = tab[i];
         e != null;
         e = tab[i = nextIndex(i, len)]) {
        ThreadLocal<?> k = e.get();
        // 说明该key已经存在, 则覆盖旧值
        if (k == key) {
            e.value = value;
            return;
        }

        /**
         * table[i]上的key为空, 说明被回收了(上面的弱引用中提到过)。
         * 这个时候说明改table[i]可以重新使用, 用新的key-value将其替换, 并删除其他无效
         的entry
         */
        if (k == null) {
            replaceStaleEntry(key, value, i);
            return;
        }
    }
    // 该索引位置上没有元素, 则新建Entry
    tab[i] = new Entry(key, value);
    int sz = ++size;
    // 不需要清理 空哈希槽或者槽key为null的, 并且大于等于扩容值, 则进行rehash,
    if (!cleanSomeSlots(i, sz) && sz >= threshold)
        // 默认16, 以2的倍数扩容
        rehash();
}

/**java
/**
 * 获取环形数组的下一个索引
 */
private static int nextIndex(int i, int len) {
    return ((i + 1 < len) ? i + 1 : 0);
}

/**

```

```
    * 获取环形数组的上一个索引
    */
    private static int prevIndex(int i, int len) {
        return ((i - 1) >= 0) ? i - 1 : len - 1;
    }
}
```

replaceStaleEntry(ThreadLocal<?> key, Object value, int staleSlot)

// 好复杂, 看看网上文章,以后深入了解吧

[ThreadLocal源码分析 - 简书 \(jianshu.com\)](#)

3.2 ThreadLocal中的get()

```

public T get() {
    //同set方法类似获取对应线程中的ThreadLocalMap实例
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    if (map != null) {
        ThreadLocalMap.Entry e = map.getEntry(this);
        if (e != null) {
            @SuppressWarnings("unchecked")
            T result = (T)e.value;
            return result;
        }
    }
    //为空返回初始化值
    return setInitialValue();
}
/**
 * 初始化设值的方法，可以被子类覆盖。
 */
protected T initialValue() {
    return null;
}

private T setInitialValue() {
    //获取初始化值，默认为null(如果没有子类进行覆盖)
    T value = initialValue();
    Thread t = Thread.currentThread();
    ThreadLocalMap map = getMap(t);
    //不为空不用再初始化，直接调用set操作设值
    if (map != null)
        map.set(this, value);
    else
        //第一次初始化，createMap在上面介绍set()的时候有介绍过。
        createMap(t, value);
    return value;
}

```

ThreadLocalMap中的getEntry()

```

private ThreadLocal.ThreadLocalMap.Entry getEntry(ThreadLocal<?> key) {
    //根据key计算索引，获取entry
    int i = key.threadLocalHashCode & (table.length - 1);
    ThreadLocal.ThreadLocalMap.Entry e = table[i];
    if (e != null && e.get() == key)
        return e;
    else
        return getEntryAfterMiss(key, i, e);
}

/**
 * 通过直接计算出来的key找不到对于的value的时候适用这个方法。
 */
private ThreadLocal.ThreadLocalMap.Entry getEntryAfterMiss(ThreadLocal<?> key, int
i, ThreadLocal.ThreadLocalMap.Entry e) {
    ThreadLocal.ThreadLocalMap.Entry[] tab = table;
    int len = tab.length;

    while (e != null) {
        ThreadLocal<?> k = e.get();
        if (k == key)
            return e;
        if (k == null)
            //清除无效的entry
            expungeStaleEntry(i);
        else
            //基于线性探测法向后扫描
            i = nextIndex(i, len);
        e = tab[i];
    }
    return null;
}

```

3.3 ThreadLocalMap中的remove()

```

private void remove(ThreadLocal<?> key) {
    ThreadLocal.ThreadLocalMap.Entry[] tab = table;
    int len = tab.length;
    //计算索引
    int i = key.threadLocalHashCode & (len-1);
    //进行线性探测，查找正确的key
    for (ThreadLocal.ThreadLocalMap.Entry e = tab[i];
        e != null;
        e = tab[i = nextIndex(i, len)]) {
        if (e.get() == key) {
            //调用weakreference的clear()清除引用
            e.clear();
            //连续段清除
            expungeStaleEntry(i);
            return;
        }
    }
}

```

4. 内存泄露

4.1 ThreadLocal 内存泄漏的原因

从上图中可以看出，hreadLocalMap使用ThreadLocal的弱引用作为key，如果一个ThreadLocal不存在外部**强引用**时，Key(ThreadLocal)势必会被GC回收，这样就会导致ThreadLocalMap中key为null，而value还存在着强引用，只有thead线程退出以后,value的强引用链条才会断掉。

但如果当前线程再迟迟不结束的话，这些key为null的Entry的value就会一直存在一条强引用链：

```
Thread Ref -> Thread -> ThreaLocalMap -> Entry -> value
```

永远无法回收，造成内存泄漏。，**所以泄露的是value值**

4.2 ThreadLocal正确的使用方法

1. 每次使用完ThreadLocal都调用它的remove()方法清除数据

其实调用set()和get() 都有可能也会清除数据

ThreadLocal会在以下过程中清理过期节点：

1. 调用set()方法时，采样清理、全量清理，扩容时还会继续检查。
2. 调用get()方法，没有直接命中，向后环形查找时。
3. 调用remove()时，除了清理当前Entry，还会向后继续清理。

2. 使用ThreadLocal时，一般建议将其声明为static final的，避免频繁创建ThreadLocal实例。

4.3 总结

由于Thread中包含变量ThreadLocalMap，因此ThreadLocalMap与Thread的生命周期是一样长，如果都没有手动删除对应key，都会导致内存泄漏。

但是使用**弱引用**可以多一层保障：弱引用ThreadLocal不会内存泄漏，对应的value在下一次ThreadLocalMap调用set(),get(),remove()的时候会被清除。

因此，ThreadLocal内存泄漏的根源是：由于ThreadLocalMap的生命周期跟Thread一样长，如果没有手动删除对应key就会导致内存泄漏，而不是因为弱引用。

[ThreadLocal的内存泄露？什么原因？如何避免？ - 知乎 \(zhihu.com\)](#)

5. 使用场景

5.1 场景一：代替参数的显式传递

当我们在写API接口的时候，通常Controller层会接受来自前端的入参，当这个接口功能比较复杂的时候，可能我们调用的Service层内部还调用了 很多其他的很多方法，通常情况下，我们会在每个调用的方法上加上需要传递的参数。

但是如果我们将参数存入ThreadLocal中，那么就不用显式的传递参数了，而是只需要ThreadLocal中获取即可。

这个场景其实使用的比较少，一方面显式传参比较容易理解，另一方面我们可以将多个参数封装为对象去传递。

5.2 场景二：解决线程安全问题

在Spring的Web项目中，我们通常会将业务分为Controller层，Service层，Dao层，我们都知道@Autowired注解默认使用单例模式，那么不同请求线程进来之后，由于Dao层使用单例，那么负责数据库连接的Connection也只有一个，如果每个请求线程都去连接数据库，那么就会造成线程不安全的问题，Spring是如何解决这个问题的呢？

在Spring项目中Dao层中装配的Connection肯定是线程安全的，其解决方案就是采用ThreadLocal方法，当每个请求线程使用Connection的时候，都会从ThreadLocal获取一次，如果为null，说明没有进行过数据库连接，连接后存入ThreadLocal中，如此一来，每一个请求线程都保存有一份自己的Connection。于是便解决了线程安全问题

ThreadLocal在设计之初就是为解决并发问题而提供一种方案，每个线程维护一份自己的数据，达到线程隔离的效果。

5.3 场景三：全局存储用户信息

在现在的系统设计中，前后端分离已基本成为常态，分离之后如何获取用户信息就成了一件麻烦事，通常在用户登录后，用户信息会保存在Session或者Token中。这个时候，我们如果使用常规的手段去获取用户信息会很费劲，拿Session来说，我们要在接口参数中加上HttpServletRequest对象，然后调用getSession方法，且每一个需要用户信息的接口都要加上这个参数，才能获取Session，这样实现就很麻烦了。

在实际的系统设计中，我们肯定不会采用上面所说的这种方式，而是使用ThreadLocal，我们会选择在拦截器的业务中，获取到保存的用户信息，然后存入ThreadLocal，那么当前线程在任何地方如果需要拿到用户信息都可以使用ThreadLocal的get()方法

(异步程序中ThreadLocal是不可靠的, 因为threadLocal是线程级别的, 所以只要开了新线程都会丢失信息)

例如, 使用了**线程池**, future, fegin调用 就会丢失threadlocal里的数据(除非threadLocal是全局的)

6. 实战

6.1 代码

具体实现流程：

- 在登录业务代码中，当用户登录成功时，生成一个登录凭证存储到redis中，将凭证中的字符串保存在cookie中返回给客户端。

- 使用一个拦截器拦截请求，从cookie中获取凭证字符串与redis中的凭证进行匹配，获取用户信息，将用户信息存储到ThreadLocal中，在本次请求中持有用户信息，即可在后续操作中使用到用户信息。

定义工具类操作[ThreadLocal] (存放，获取，删除用户信息)

```
public class ThreadLocalUtil {

    /**
     * 保存用户对象的ThreadLocal 在拦截器操作 添加、删除相关用户数据
     */
    private static final ThreadLocal<FeginUser> userThreadLocal = new
ThreadLocal<FeginUser>();

    /**
     * 添加当前登录用户方法 在拦截器方法执行前调用设置获取用户
     * @param user
     */
    public static void addCurrentUser(FeginUser user){
        userThreadLocal.set(user);
    }

    /**
     * 获取当前登录用户方法
     */
    public static FeginUser getCurrentUser(){
        return userThreadLocal.get();
    }

    /**
     * 删除当前登录用户方法 在拦截器方法执行后 移除当前用户对象
     */
    public static void remove(){
        userThreadLocal.remove();
    }
}
```

拦截器

```

@Component
@Slf4j
public class UserInfoInterceptor implements HandlerInterceptor {

    @Autowired
    private UserInfoUtil userInfoUtil;

    /**
     * 请求执行前执行的，将用户信息放入ThreadLocal
     * @param request
     * @param response
     * @param handler
     * @return
     * @throws Exception
     */
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response,
Object handler) throws Exception {
        FeginUser user;
        try{
            user = userInfoUtil.getUser(request);
        }catch (CustomException e){
            log.info("*****用户未登录， ThreadLocal无信息
*****");
            return true;
        }
        if (null!=user) {
            log.info("*****用户已登录，用户信息放入
ThreadLocal*****");
            ThreadLocalUtil.addCurrentUser(user);
            return true;
        }
        log.info("*****用户未登录， ThreadLocal无信息
*****");
        return true;
    }

    /**
     * 接口访问结束后，从ThreadLocal中删除用户信息
     * @param request
     * @param response
     * @param handler
     * @param ex
     * @throws Exception
     */
    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler, Exception ex) throws Exception {
        log.info("*****接口调用结束， 从ThreadLocal删除用户信息
*****");
    }
}

```

```
ThreadLocalUtil.remove();  
}
```

6.2 异常处理

6.2.1 feign调用丢失请求头

从请求头中获取登录token, 会有个问题, 如果上游是feign调用, 则请求头会丢失, 所以需要在feign调用时手动设置token, **因为feign调用会使用新的http请求且不会携带原来http的header信息**

```
@Bean  
public RequestInterceptor requestInterceptor() {  
    //处理feign远程调用丢失请求头问题  
    return template -> {  
        HttpRequestUtil.getHttpHeader(Constant.AUTHORIZATION)  
            .ifPresent(auth -> template.header(Constant.AUTHORIZATION,  
auth));  
        HttpRequestUtil.getHttpHeader(Constant.POWER_MENU_ID)  
            .ifPresent(auth -> template.header(Constant.POWER_MENU_ID,  
auth));  
        HttpRequestUtil.getHttpHeader(Constant.X_FLAG)  
            .ifPresent(auth -> template.header(Constant.X_FLAG, auth));  
        HttpRequestUtil.getHttpHeader(Constant.PLATFORM_FLAG)  
            .ifPresent(auth -> template.header(Constant.PLATFORM_FLAG,  
auth));  
    };  
}
```

6.2.2 多线程中丢失请求头

因为threadLocal是线程内部的, 使用多线程后threadLocal会不再有数据

解决方案:

1. 手动为子线程里的request设置请求头
2. 临时存储方案, 将threadLocal中的数据在子线程中再设置一遍

[获取用户信息工具类](#)

spring security 框架默认也没有解决

[spring security 如何在子线程中获取父线程中的用户认证信息（更改安全策略）](#) -
[precedeforetime - 博客园 \(cnblogs.com\)](#)

7. 扩展

7.1 InheritableThreadLocal

InheritableThreadLocal类继承并重写了ThreadLocal的3个函数：

```
public class InheritableThreadLocal<T> extends ThreadLocal<T> {

    /**
     * 该函数在父线程创建子线程，向子线程复制InheritableThreadLocal变量时使用
     */
    protected T childValue(T parentValue) {
        return parentValue;
    }

    /**
     * 由于重写了getMap，操作InheritableThreadLocal时，
     * 将只影响Thread类中的inheritableThreadLocals变量，
     * 与threadLocals变量不再有关系
     */
    ThreadLocalMap getMap(Thread t) {
        return t.inheritableThreadLocals;
    }

    /**
     * 类似于getMap，操作InheritableThreadLocal时，
     * 将只影响Thread类中的inheritableThreadLocals变量，
     * 与threadLocals变量不再有关系
     */
    void createMap(Thread t, T firstValue) {
        t.inheritableThreadLocals = new ThreadLocalMap(this, firstValue);
    }

}
```

线程间传值实现原理

线程初始化时将数据从 inheritableThreadLocals 取出并设置

thread类

```

public class Thread implements Runnable {
    .....(其他源码)
    /*
     * 当前线程的ThreadLocalMap，主要存储该线程自身的ThreadLocal
     */
    ThreadLocal.ThreadLocalMap threadLocals = null;

    /*
     * InheritableThreadLocal，自父线程集成而来的ThreadLocalMap，
     * 主要用于父子线程间ThreadLocal变量的传递
     * 本文主要讨论的就是这个ThreadLocalMap
     */
    ThreadLocal.ThreadLocalMap inheritableThreadLocals = null;
    .....(其他源码)
}

```

线程初始化

```

/**
 * 默认情况下，设置inheritThreadLocals可传递
 */
private void init(ThreadGroup g, Runnable target, String name,
                  long stackSize) {
    init(g, target, name, stackSize, null, true);
}
/**
 * 初始化一个线程。
 * 此函数有两处调用，
 * 1、上面的 init()，不传AccessControlContext，inheritThreadLocals=true
 * 2、传递AccessControlContext，inheritThreadLocals=false
 */
private void init(ThreadGroup g, Runnable target, String name,
                  long stackSize, AccessControlContext acc,
                  boolean inheritThreadLocals) {
    .....（其他代码）

    if (inheritThreadLocals && parent.inheritableThreadLocals != null)
        this.inheritableThreadLocals =
            ThreadLocal.createInheritedMap(parent.inheritableThreadLocals);

    .....（其他代码）
}

```

所以只有线程创建时才会 执行值传递

7.2 TransmittableThreadLocal(TTL)

阿里提供的 在使用线程池等会池化复用线程的执行组件情况下, 提供 `ThreadLocal` 值的传递功能, 解决异步执行时上下文传递的问题。

其底层是 将数据从父线程中取出来, 再手动设置到子线程中

[TransmittableThreadLocal \(TTL\)](#)

[ThreadLocal, 一篇文章就够了 - 知乎 \(zhihu.com\)](#)

[ThreadLocal源码分析 - 简书 \(jianshu.com\)](#)

[ThreadLocal为什么会导内存泄漏? - Chen洋 - 博客园 \(cnblogs.com\)](#)

[Java-ThreadLocal三种使用场景_用心去追梦的博客-CSDN博客_java threadlocal场景](#)

[ThreadLocal存放用户信息 \(springboot\) _神都燕的博客-CSDN博客_threadlocal存储用户信息](#)

[InheritableThreadLocal详解 - 简书 \(jianshu.com\)](#)

[ThreadLocal 面试夺命11连问_Young丶的博客-CSDN博客](#)